

MICROSOFT AZURE · CONSULTANT REPORT

CIVICPULSE

Civic Incident Data Platform for a Victorian Local Council

PREPARED BY	Ayush Khatri
STUDENT ID	22727228
SUBJECT	BUS5001 — Cloud Platforms and Analytics
ASSESSMENT	Assessment 1 — Adopting Cloud Platforms
INSTITUTION	La Trobe University
DATE	11 August 2026

REQUIRED LINKS

PUBLIC DASHBOARD	https://stcivicpulseprodaue001.z8.web.core.windows.net/
SOURCE FILES	https://github.com/khatriayush123-cloud/civicpulse-bus5001
DEMONSTRATION VIDEO	A1 video.mp4

1. OVERVIEW

CivicPulse is a minimum viable civic incident platform for a small Victorian council. Incident files land in private storage, a Logic App validates and de-identifies each record and publishes a single JSON file, and a static website renders it for residents. This report covers the governance model, storage design, workflow logic, dashboard and design rationale.

2. GOVERNANCE APPROACH

The binding constraint is not budget but attention. The council runs a small generalist IT team, not a cloud practice, so controls needing constant vigilance decay while controls enforced by the platform do not. Every decision below follows from that.

- **Naming.** Resources follow the Cloud Adoption Framework pattern `[type]-[workload]-[environment]-[region]-[instance]`, giving `rg-civicpulse-prod-aue-001`. An officer can tell what a resource is and which environment it belongs to without opening documentation, and consistent prefixes make scripted maintenance safe enough to write.
- **Tagging.** Four mandatory tags — `CostCenter`, `Environment`, `DataClassification` and `Owner` — are enforced by Azure Policy with a Deny effect. `CostCenter` makes spend reconcilable against the budget, `Environment` drives production resource locks, and `Owner` is a role mailbox rather than a person, so accountability survives staff turnover.
- **Classification and access.** Each container carries exactly one classification, so the control applied follows from where data sits rather than from judgement. Access is granted through Entra ID groups using Azure RBAC at container scope.

ROLE	WHO	ACCESS
Administrative	Council IT (Entra ID group)	Contributor on the resource group; cannot assign roles
Operational	Logic App managed identity	Storage Blob Data Contributor, per container
Read-only	Analysts and IT support	Reader on curated-data, rejected-data and logs
Public	Residents and councillors	Anonymous read of \$web only

- **Least privilege.** Contributor and Owner are operationally identical except that Contributor cannot create role assignments, so a compromised admin account cannot grant an attacker lasting access. Workflows use a system-assigned managed identity, so no key exists to leak, and no human identity has write access to any container.
- **Region, retention and cost.** Everything runs in Australia East, paired with Australia Southeast, so data never leaves the country. Lifecycle rules tier ageing data automatically, and budget alerts at 50, 80 and 100 per cent of forecast give warning while remediation is cheap.

3. STORAGE DESIGN

Six containers separate the estate by classification. `raw-data` holds the original file, never edited — the ground truth every published figure traces back to. `curated-data` holds validated, de-identified records. `rejected-data` quarantines failing rows with the reason. `archive` keeps a dated copy, `processing-logs` a per-run audit record, and `$web` the dashboard and single public JSON file.

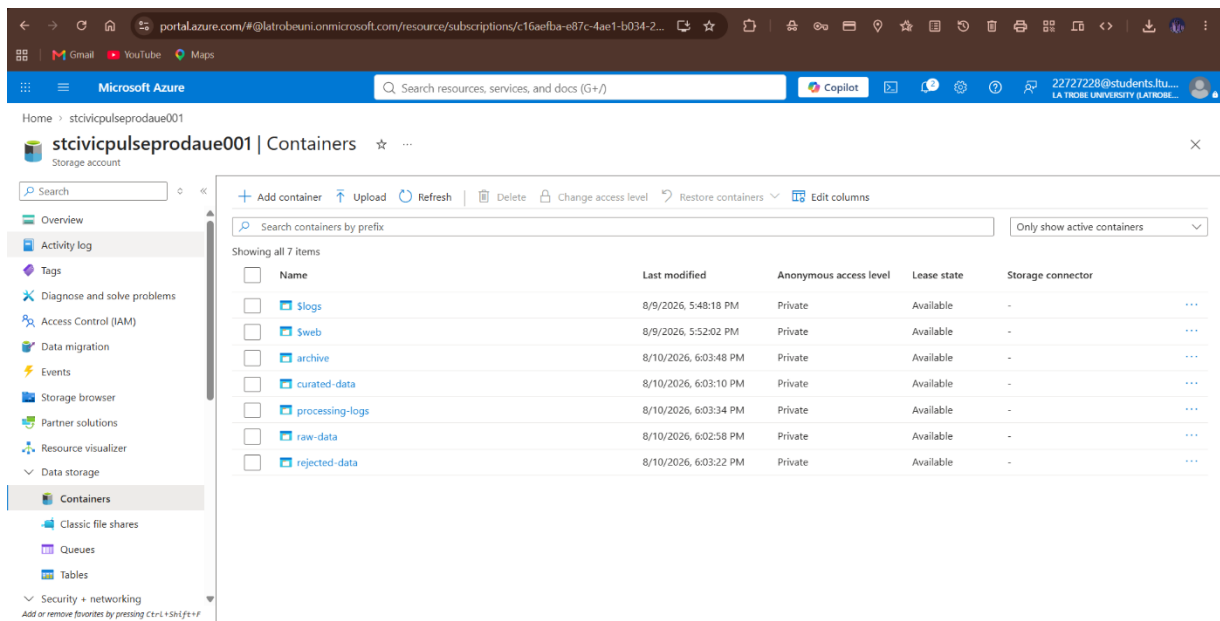


Figure 1 — Container topology. Every operational container is private; only \$web allows anonymous read.

The public boundary is structural, not procedural. The dashboard never queries a container holding personal information; it fetches one pre-computed file. Lifecycle rules move raw data to Cool at thirty days, matching the tier's minimum so no early-deletion charge applies, and to Archive at ninety, beyond the window in which a raw file is needed quickly.

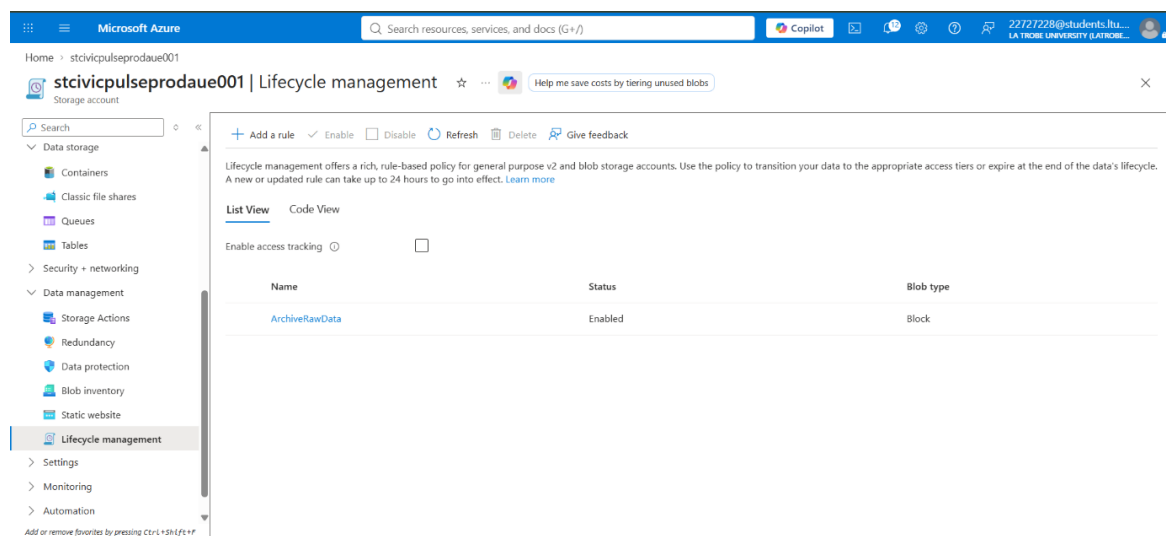


Figure 2 — Lifecycle rule tiering ageing data automatically, with no manual intervention.

The structure also supports automation: the workflow reads and writes by container path under one identity, so no step depends on a person moving a file. Traceability follows from the structure. Every record keeps its **incident_id** and each zone is partitioned by date, so a published figure traces back to the original submission, and a suspiciously low one is explained by checking the matching date in **rejected-data**.

4. WORKFLOW AND SHAPING LOGIC

A blob trigger on **raw-data** starts the Logic App within a minute of upload. It decodes and parses the file, then validates every row against six rules: non-empty **incident_id**, **suburb** and **category**; a **reported_date** matching YYYY-MM-DD; and **status** and **priority** from the approved lists. Validation uses two Filter array actions rather than a loop — one billed action regardless of row count.

Invalid rows are not discarded but written to **rejected-data** with their original text and failure reason, so incompleteness is countable and IT can correct and resubmit rather than asking a resident to report something twice.

Public-field decisions

The mapping step reads only six of the nine columns. **description**, **lat** and **lon** are never referenced, so they cannot exist downstream. Free text written by the public routinely contains names and phone numbers and cannot be reliably sanitised, and a coordinate at submission precision identifies a household. Excluding them in the pipeline rather than hiding them in the dashboard means no front-end change can expose them.

Calculations

Every metric is computed at runtime. Category and priority counts derive from the values actually present using a **union** expression rather than a fixed list, so a new category appears automatically. Recent records use `take(reverse(sort(..., 'reported_date')), 10)`, correct only because the date rule enforces ISO format, where alphabetical order is chronological order. Nothing is hard-coded.

Processing sits inside a **ScopeTry**. A **ScopeCatch** set to run on failure captures `result('ScopeTry')`, writes a failure log and emails IT. The workflow also archives the source file and writes an audit record every run.

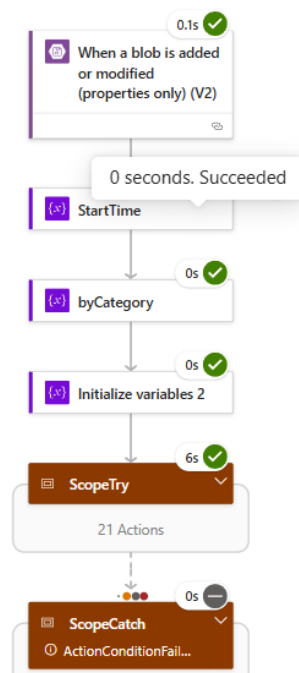


Figure 3 — A successful run: **ScopeTry** completes 21 actions and **ScopeCatch** is correctly skipped.

Test evidence

Clean data proves the arithmetic but not that invalid records are stopped, so I built a test file with five defects, one per rule: a date written 14/03/2026, a status of Pending, a priority of Urgent, and two empty required fields. All five were rejected and preserved verbatim; the remaining fifty-five published normally.

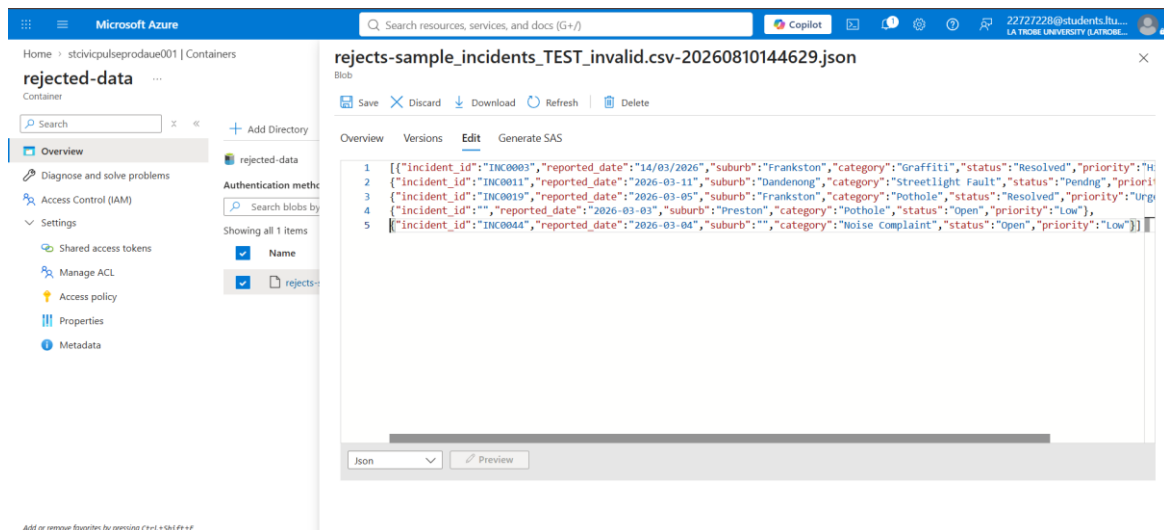


Figure 4 — The five quarantined records, each keeping its original malformed value.

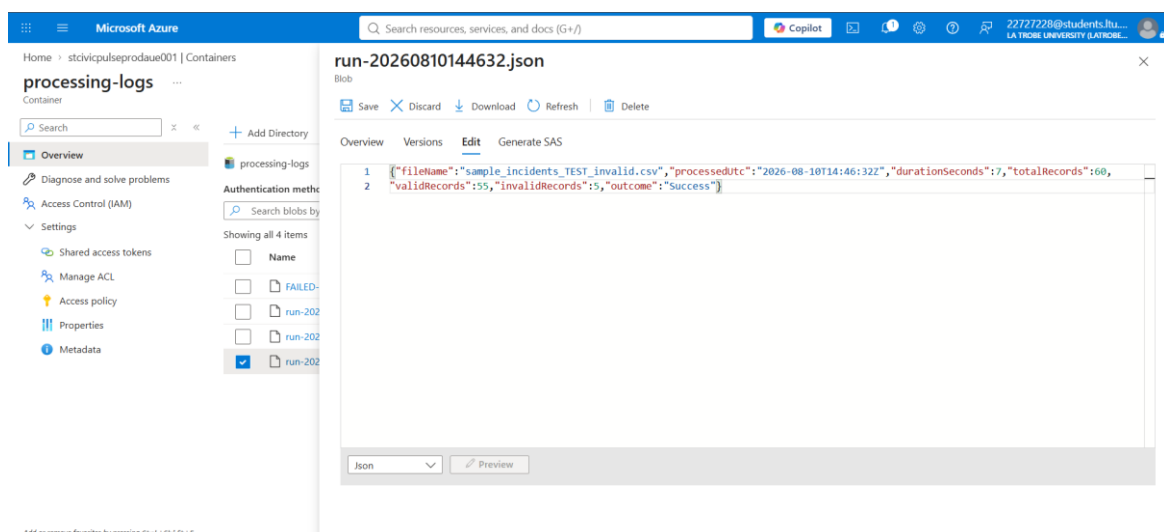


Figure 5 — Audit record for the same run: 60 read, 55 valid, 5 rejected.

Both runs reconcile — sixty read minus five rejected equals fifty-five published, and status counts sum to the total each time. I calculated the expected figures from the source file beforehand, so output was checked against a known answer, not just inspected.

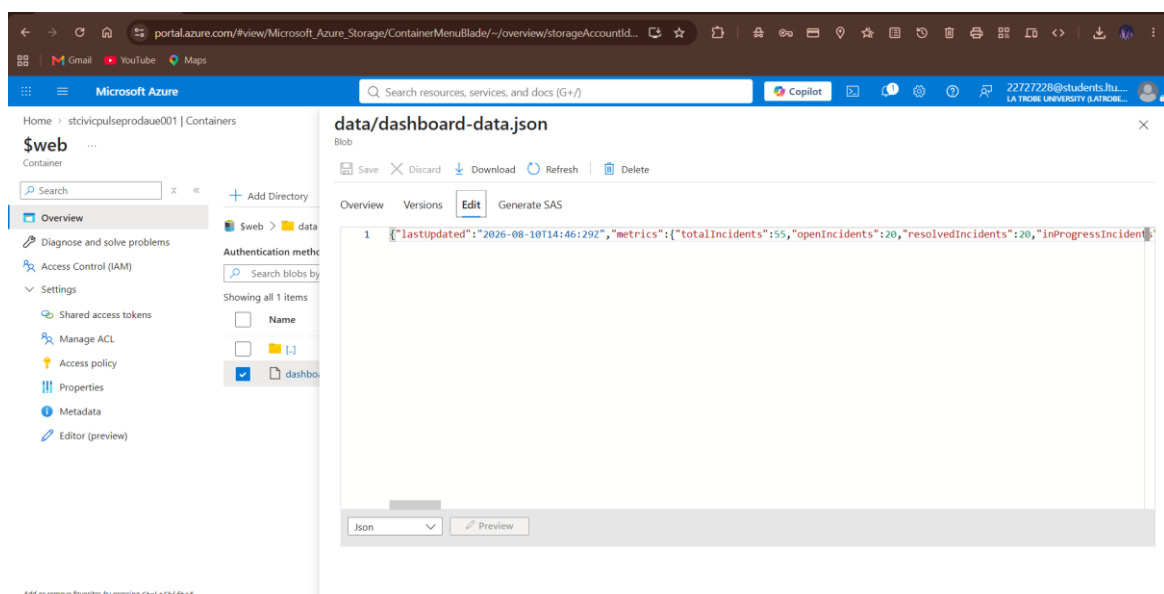


Figure 6 — The generated dashboard-data.json, showing counts emitted as numbers rather than strings.

5. PUBLIC DASHBOARD

The dashboard is a static site in \$web — index.html and app.js, no server, no database. It fetches dashboard-data.json and shows four KPI values, a bar chart toggling between category and priority, a table of the ten most recent records, and the lastUpdated timestamp converted from UTC to local time.

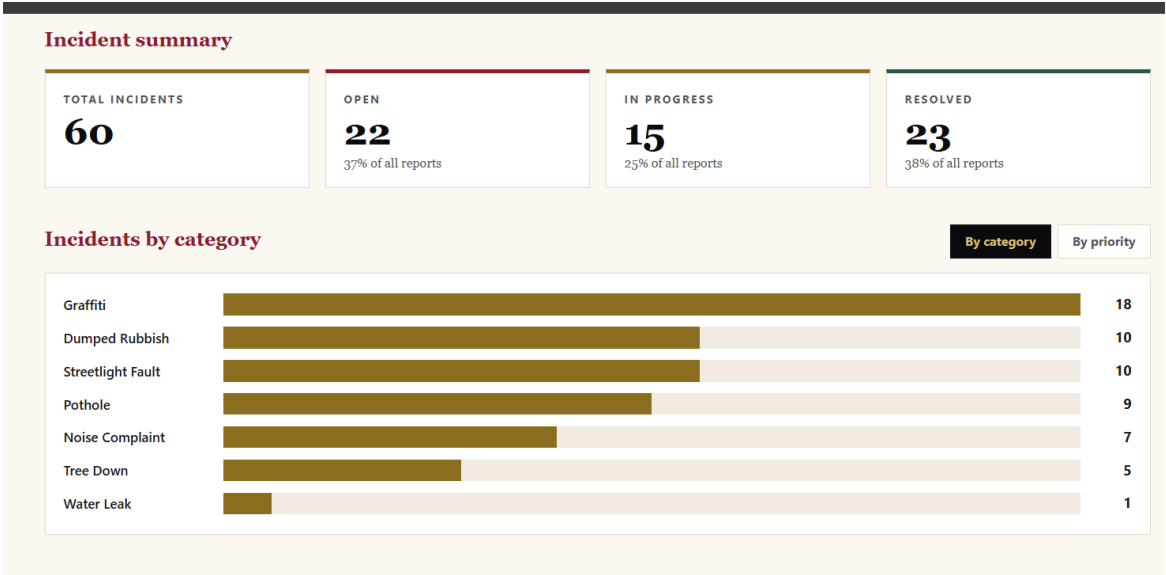


Figure 7 — KPIs and the category chart. Bars and labels are generated from the JSON.

It calculates nothing the contract requires. The only arithmetic is presentational: percentage shares under the KPI cards and bar widths relative to the largest value. Categories, labels and colours all derive from the published file.

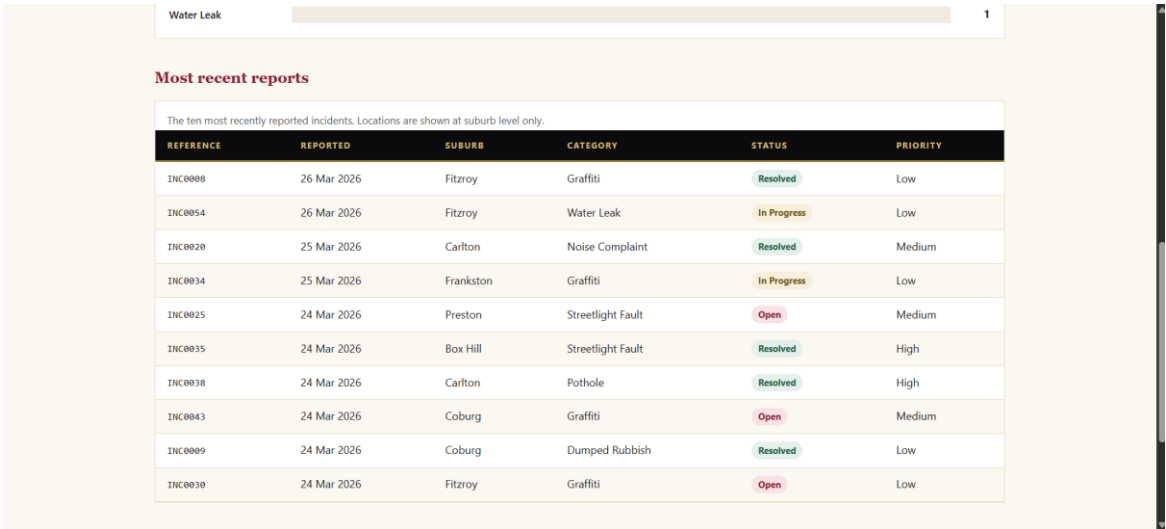


Figure 8 — Recent records. Only the six approved fields appear; there is no description or coordinate column.

Three states are handled: loading, an error panel with a retry, and an empty state if a file yields no valid records. The page refreshes once a minute, so it reflects a new run untouched. It uses semantic headings, a skip link and a text alternative for the chart, and reflows to one column on a phone. A footer explains that descriptions and coordinates are removed and locations shown at suburb level.

6. ARCHITECTURE AND DESIGN RATIONALE

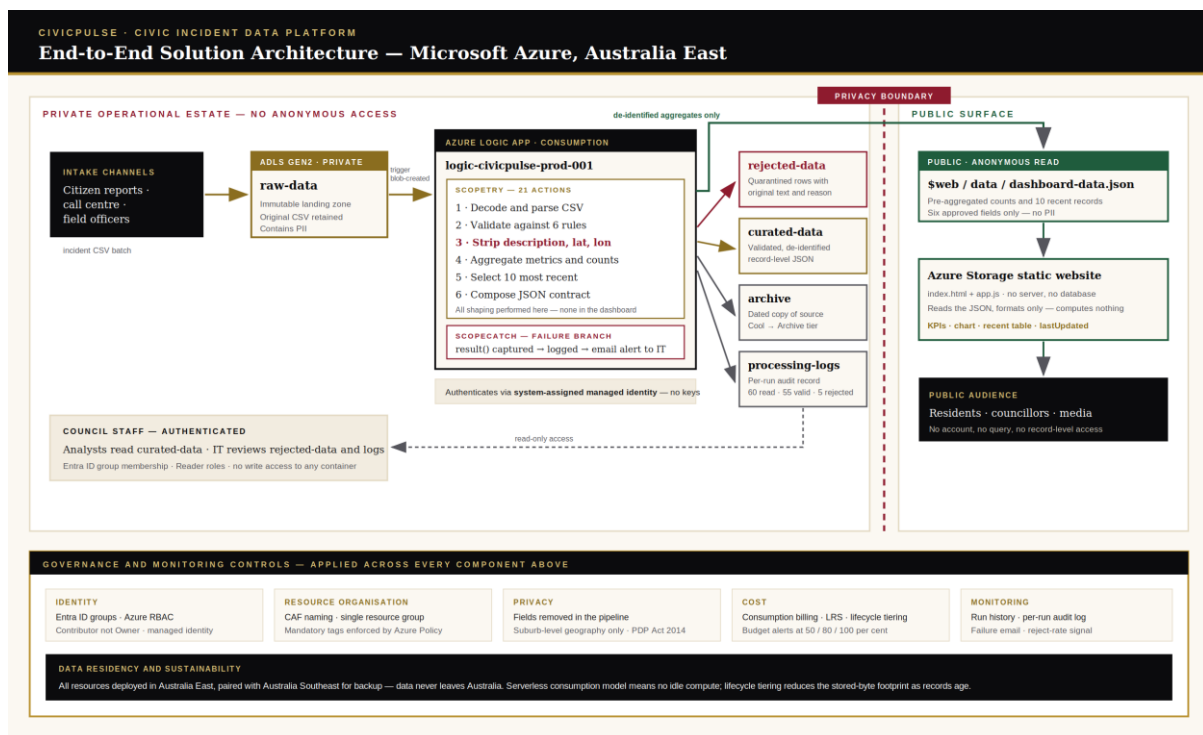


Figure 9 — End-to-end architecture. The dashed line marks the only point where data becomes publicly readable.

Four decisions carried most weight. Logic Apps over Functions, because the logic is readable by non-developers and run history doubles as an audit trail. A blob trigger over a schedule, because the requirement is automatic detection. Static hosting over App Service, because there is no compute to patch. Managed identity over account keys, because a credential that does not exist cannot leak.

Governance shaped the build directly. The naming standard determined resource organisation; the classification scheme determined the container topology, since each container carries one classification; RBAC groups and managed identity determined access, with workflows writing and staff only reading; the privacy position determined which fields the mapping step reads; and the cost controls determined the redundancy tier and the lifecycle rules.

Cost profile

Nothing bills at rest. The Logic App charges per action, roughly thirty per file, so a few files a day costs very little monthly. Storage is tens of gigabytes a year, most falling into Cool and Archive as it ages, and the static site costs storage plus trivial egress. The real cost risk is unbounded log ingestion, which is why log retention is a deliberate setting.

Limitations

Columns are read by position, so an inserted column upstream would silently misalign fields. Validation is structural, not semantic — a suburb is checked for presence, not against a gazetted list. There is no de-duplication across batches. LRS protects against disk failure but not loss of a region, and publication overwrites in place rather than atomically.

Improvements

Parsing by header name, duplicate detection on `incident_id`, and an Event Grid trigger instead of polling are inexpensive and would come first. Beyond that: a separate storage account for `$web` so anonymous access can be disabled account-wide, Power BI over `curated-data`, and Bicep templates for reproducibility. The naming, tagging and identity model already accommodate a second department without redesign.

7. REFLECTION ON GENERATIVE AI USE

I used generative AI as a design and debugging partner rather than an author. It helped me structure the governance argument, draft Logic Apps expressions, and write the dashboard code.

One example stands out. The AI produced a parsing expression that split the file using `body('GetBlob')` directly. The run failed with an `InvalidTemplate` error saying `replace` had been given an `Object` rather than a string. Reading the run history, I found the connector returns a wrapper object holding base64 content, not plain text, and corrected the expression to decode it first. A second fault surfaced once parsing worked: every record failed validation because the file used Windows line endings, leaving a carriage return on the last field, so I added a `replace` to strip it.

The limitation this exposed is that the AI generated confident, plausible code that did not match the connector's actual behaviour, because it could not know the runtime shape of the data. Nothing it produced was trustworthy until executed and checked. I verified every published figure against the source file before accepting the output.